

Investigación — Skip List (Lista de Saltos)

Curso: Algoritmos y Estructuras de Datos — SIS210 **Práctica:** N.º 05 — Listas Enlazadas **Estudiante:** **Docente:** Dr. Aldo Hernán Zanabria Gálvez **Fecha:** 02/10/2026

1. Introducción

La **Skip List** (lista de saltos) fue propuesta por **William Pugh en 1990** como una alternativa **probabilística** a los árboles balanceados. Su objetivo es ofrecer las mismas garantías de complejidad que un árbol AVL o un árbol B (búsqueda, inserción y eliminación en $O(\log n)$ esperado), pero con una implementación **mucho más simple** y con operaciones **más fáciles de paralelizar** y de razonar.

La idea central es agregar **niveles jerárquicos de punteros** sobre una lista enlazada simple ordenada, de manera que algunos nodos "salten" por encima de otros y permitan descartar grandes porciones de la lista en cada paso. A diferencia de los árboles, **no hay rotaciones ni rebalanceos**: la estructura se mantiene balanceada de forma **aleatoria** gracias a un generador de números pseudoaleatorios.

Este documento:

1. Describe la estructura de una Skip List.
2. Explica cómo logra $O(\log n)$ esperado.
3. Compara con lista enlazada simple, árbol AVL y árbol B.
4. Presenta un diagrama propio.
5. Cierra con conclusiones y referencias en APA 7.

2. Estructura de una Skip List

2.1 Idea intuitiva

Imaginemos una lista enlazada simple ordenada de enteros:

```
cabeza → [1] → [3] → [4] → [6] → [7] → [9] → [10] → [13] → [14] → [17] → NULL
```

Para buscar el valor **14**, hay que recorrer **9 nodos** $\rightarrow O(n)$.

Ahora agreguemos una **"vía rápida"** con punteros que salten cada dos nodos:

```
Nivel 2: cabeza → [4] → [9] → [14] → NULL
Nivel 1: cabeza → [3] → [4] → [7] → [9] → [13] → [14] → NULL
Nivel 0: cabeza → [1] → [3] → [4] → [6] → [7] → [9] → [10] → [13] → [14] → [17] → NULL
```

Ahora buscar **14**:

- En el nivel 2: **cabeza** → 4 → 9 → 14 (3 saltos).
- Se descarta toda la primera mitad de la lista en un solo salto.

Al agregar **más niveles**, la búsqueda se acelera aún más, de forma análoga a la búsqueda binaria.

2.2 Definición formal

Una Skip List es una colección de listas enlazadas ordenadas $S_0, S_1, \dots, S_h$ (donde S_0 es la lista base y S_h es el nivel más alto). Cada lista S_i contiene un subconjunto de los elementos de S_{i-1} , y todo elemento de S_i también aparece en S_{i-1} .

Propiedades:

- S_0 contiene **todos** los elementos.
- Cada S_i (con $i > 0$) contiene, en promedio, **la mitad** de los elementos de S_{i-1} .
- La probabilidad de que un nodo aparezca en el nivel $i + 1$ dado que aparece en el nivel i es p (típicamente $p = 0.5$).
- El número esperado de niveles es $O(\log n)$.

2.3 Inserción aleatoria de niveles

Cuando se inserta un nuevo elemento:

1. Se busca su posición en S_0 (como en una lista enlazada ordenada).
2. Se lanza una moneda (o se genera un número aleatorio):
 - Con probabilidad p se promueve al nivel 1.
 - Con probabilidad p se promueve al nivel 2.
 - Y así sucesivamente hasta que la moneda salga "cruz" o se alcance el nivel máximo.

Pseudocódigo simplificado:

```
def nivel_aleatorio(p=0.5, max_nivel=32):
    nivel = 0
    while random() < p and nivel < max_nivel:
        nivel += 1
    return nivel
```

En promedio, un nodo tiene $1/(1 - p)$ niveles. Con $p = 0.5$, cada elemento tiene en promedio 2 niveles.

3. ¿Cómo logra $O(\log n)$ esperado?

3.1 Búsqueda

Para buscar un valor x :

1. Comenzar en el nivel más alto, en el nodo cabeza.
2. En cada nivel, avanzar mientras **siguiente.dato** < x .
3. Cuando no se pueda avanzar (o el siguiente sea mayor o igual que x), bajar un nivel.

4. Repetir hasta llegar al nivel 0.
5. En el nivel 0, verificar si el siguiente nodo contiene x .

Ejemplo con la Skip List anterior, buscando el valor 14:

Paso	Nivel	Nodo actual	Acción
1	2	cabeza	Avanzar a 4 ($4 < 14$)
2	2	4	Avanzar a 9 ($9 < 14$)
3	2	9	Avanzar a 14 ($14 = 14$) 

Total: 3 saltos para buscar en una lista de 10 elementos.

3.2 Análisis probabilístico

En cada nivel, la probabilidad de subir es p y de bajar es $1 - p$. El número esperado de niveles es $\log_{1/p}(n)$. Con $p = 0.5$:

$$\text{Nivel esperado} = \log_2(n)$$

En cada nivel se hacen, en promedio, $1/(1 - p) = 2$ comparaciones. Por lo tanto, el número esperado de comparaciones totales es:

$$O(2 \cdot \log_2(n)) = O(\log n)$$

3.3 Inserción y eliminación

- **Insertar:** buscar la posición en S_0 ($O(\log n)$) y luego insertar el nodo en cada nivel donde aparezca ($O(\log n)$ niveles esperados).
- **Eliminar:** buscar el nodo ($O(\log n)$) y desenlazarlo de cada nivel donde aparezca ($O(\log n)$ niveles esperados).

Ambas operaciones mantienen un coste de $O(\log n)$ **esperado**.

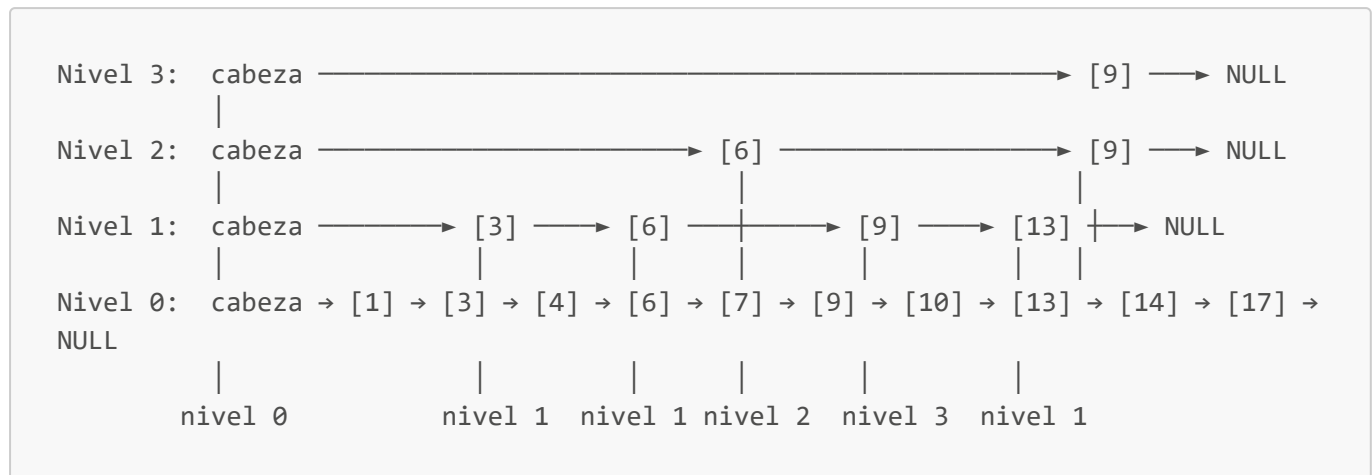
3.4 Complejidades

Operación	Lista simple	Skip List (esperado)	Skip List (peor caso)
Búsqueda	$O(n)$	$O(\log n)$	$O(n)$
Inserción	$O(n)$	$O(\log n)$	$O(n)$
Eliminación	$O(n)$	$O(\log n)$	$O(n)$
Espacio	$O(n)$	$O(n)$	$O(n \log n)$

El peor caso $O(n)$ es exponencialmente improbable (probabilidad $(1/2)^n$), pero teóricamente posible.

4. Diagrama propio

A continuación, un diagrama de una Skip List con 10 elementos y 4 niveles. Cada $\rightarrow$ representa un puntero en ese nivel. Los nodos con $[]$ contienen el dato; las columnas indican los niveles donde aparece cada nodo.



Lectura del diagrama:

- El nodo 1 solo aparece en el nivel 0.
- El nodo 3 aparece en los niveles 0 y 1.
- El nodo 6 aparece en los niveles 0, 1 y 2.
- El nodo 9 aparece en los niveles 0, 1, 2 y 3 (es el nodo "más alto").
- El nodo 13 aparece en los niveles 0 y 1.
- El nodo 14 solo aparece en el nivel 0.

Cómo se lee una búsqueda de 13:

1. **Nivel 3:** cabeza $\rightarrow$ 9 (9 < 13, avanzar). No hay siguiente en nivel 3 $\rightarrow$ bajar.
2. **Nivel 2:** 9 $\rightarrow$ NULL (no hay siguiente) $\rightarrow$ bajar.
3. **Nivel 1:** 9 $\rightarrow$ 13 (13 = 13 ☒). Encontrado.

Total: 3 saltos para llegar a 13.

5. Comparación con otras estructuras del curso

5.1 Skip List vs. Lista enlazada simple

Aspecto	Lista enlazada simple	Skip List
Estructura	1 puntero por nodo	Múltiples punteros por nodo (niveles)
Búsqueda	$O(n)$	$O(\log n)$ esperado
Inserción	$O(n)$	$O(\log n)$ esperado
Eliminación	$O(n)$	$O(\log n)$ esperado
Ordenamiento	No requiere	Requiere mantener el orden secuencial
Espacio	$O(n)$	$O(n)$ esperado
Rebalanceo	No aplica	No hay rebalanceo explícito (es probabilístico)

Aspecto	Lista enlazada simple	Skip List
Implementación	Muy simple	Simple pero requiere gestión de aleatoriedad

Ventaja principal de la Skip List: conserva la flexibilidad de inserción de una lista enlazada (no hay complejas rotaciones de punteros distantes), pero dotándola de una complejidad logarítmica.

5.2 Skip List vs. Árbol AVL

Aspecto	Árbol AVL	Skip List
Búsqueda	$O(\log n)$ garantizado	$O(\log n)$ esperado
Inserción	$O(\log n)$ garantizado (con rotaciones)	$O(\log n)$ esperado (sin rotaciones)
Eliminación	$O(\log n)$ garantizado (con rotaciones)	$O(\log n)$ esperado (sin rotaciones)
Balanceo	Estricto (factor de balance ≤ 1)	Probabilístico
Espacio	1 puntero/factor extra por nodo	Múltiples punteros por nodo (según nivel)
Implementación	Compleja (múltiples casos de rotación)	Más simple y directa
Paralelización	Difícil (las rotaciones afectan subárboles)	Fácil (las mutaciones son puramente locales)
Recorrido ordenado	Requiere recorrido <i>In-order</i>	Directo y lineal recorriendo el nivel 0

Ventajas de la Skip List sobre el AVL:

- Implementación mucho más accesible y menos propensa a bugs de diseño.
- Ausencia de rotaciones globales (ideal para entornos con alta concurrencia).
- Recorrido secuencial natural e inmediato a través de su base.

Desventajas:

- No garantiza de forma dura el peor caso en $O(\log n)$.
- Consume una cantidad ligeramente superior de memoria debido a la redundancia de punteros.

5.3 Skip List vs. Árbol B

Aspecto	Árbol B	Skip List
Uso principal	Bases de datos, sistemas de archivos	Memoria principal, índices en RAM
Nodos	Grandes (múltiples claves por nodo)	Un dato por nodo

Aspecto	Árbol B	Skip List
Búsqueda	$O(\log n)$	$O(\log n)$ esperado
Altura	Muy baja ($\log_m n$)	$O(\log n)$ niveles
Localidad de caché	Excelente (nodos contiguos en bloques)	Regular (nodos dispersos en el heap)
Paralelización	Moderada (bloqueos por división/fusión)	Buena (modificaciones concurrentes locales)
Complejidad de implementación	Alta	Media

- **Ventaja del Árbol B:** Excelente localidad de caché debido a que cada nodo ocupa una página física o bloque de almacenamiento. Es la estructura óptima por excelencia para persistencia en disco.
- **Ventaja de la Skip List:** Es mucho más simple y dinámica en memoria RAM, con un soporte superior y natural para la concurrencia.

5.4 Tabla comparativa final

Estructura	Búsqueda	Inserción	Eliminación	Balanceo	Implementación
Lista enlazada simple	$O(n)$	$O(n)$	$O(n)$	No	Muy simple
Skip List	$O(\log n)$ esp.	$O(\log n)$ esp.	$O(\log n)$ esp.	Probabilístico	Simple
Árbol AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	Rotaciones	Compleja
Árbol B	$O(\log n)$	$O(\log n)$	$O(\log n)$	División de nodos	Muy compleja
Árbol Rojo-Negro	$O(\log n)$	$O(\log n)$	$O(\log n)$	Recoloreo + rotaciones	Compleja

Observación: La Skip List y el árbol AVL comparten la misma complejidad asintótica promedio ($O(\log n)$), pero la Skip List no ofrece una garantía rígida en el peor de los casos. A cambio, prescinde de complejas reestructuraciones globales, lo que reduce sustancialmente el coste de desarrollo y maximiza el paralelismo en entornos concurrentes.

6. Aplicaciones reales de las Skip Lists

- **Redis:** Las utiliza directamente para dar soporte a su tipo de datos **Sorted Sets (ZSET)**, haciendo posibles operaciones complejas como **ZADD**, **ZRANGE** y **ZRANK** en tiempos eficientes de $O(\log n)$.
- **LevelDB (Google):** Emplea una Skip List para el funcionamiento de su **MemTable**, la estructura en memoria RAM estructurada donde se acumulan las escrituras de alta velocidad antes de ser volcadas secuencialmente al disco.

- **Apache Lucene:** Utiliza estas listas internamente para acelerar las búsquedas sobre los índices invertidos de palabras clave (*postings lists*).
 - **MemSQL / SingleStore:** Las implementa para construir índices estructurados ultra rápidos directamente sobre la memoria principal.
 - **Java (`ConcurrentSkipListMap`):** Expone esta estructura en su paquete `java.util.concurrent` como un mapa ordenado perfectamente distribuido e inmune a bloqueos globales (*lock-free*).
-

7. Ventajas y desventajas

Ventajas

- **Implementación sencilla:** Es drásticamente más fácil de programar y depurar que un árbol AVL o un árbol Rojo-Negro.
- **Sin rotaciones masivas:** Las operaciones de mutación se limitan a reajustar punteros locales e inmediatos.
- **Rendimiento predecible:** Garantiza un coste asintótico esperado de $O(\log n)$ para las operaciones de búsqueda, inserción y borrado.
- **Concurrencia amigable:** Debido a que las mutaciones afectan solo a los nodos vecinos, es ideal para algoritmos concurrentes sin bloqueos amplios.
- **Recorrido secuencial inmediato:** La lista base en el nivel 0 permite recorrer todos los elementos en orden lineal de forma natural.
- **Comportamiento controlable:** Al modificar el parámetro de probabilidad p , el desarrollador puede balancear dinámicamente el consumo de memoria frente a la velocidad de salto.

Desventajas

- **Falta de garantías duras:** Aunque matemáticamente la probabilidad es despreciable, el peor caso teórico degradado de $O(n)$ existe si todas las monedas caen del mismo lado.
 - **Mayor consumo de memoria:** Almacenar múltiples capas de punteros redundantes incrementa la huella de bytes respecto a una lista simple.
 - **Dependencia aleatoria:** Requiere un generador de números pseudoaleatorios de buena calidad; una mala distribución arruinaría las propiedades de balanceo.
 - **Menos eficiente para almacenamiento secundario:** La dispersión física de los nodos en el *heap* penaliza la localidad de caché en operaciones basadas en disco, quedando por detrás del diseño estructurado por bloques de un Árbol B.
 - **No determinista:** Es imposible predecir con exactitud la estructura y altura de niveles interna que tomará la lista para un conjunto dado de entradas.
-

8. Conclusiones

1. La **Skip List** logra una eficiencia esperada de $O(\log n)$ en búsqueda, inserción y eliminación mediante múltiples niveles de punteros generados de forma probabilística.
2. Se consolida como una alternativa limpia, eficiente y elegante frente a los árboles balanceados tradicionales, con los cuales comparte las mismas métricas asintóticas promedio.
3. Destaca principalmente en escenarios que demanden una baja complejidad de código y alta disponibilidad para operaciones paralelas y concurrentes en memoria de acceso aleatorio (RAM).

4. Su diseño penaliza el peor caso teórico y la densidad de memoria, pero su adopción en plataformas críticas de la industria tecnológica moderna (como Redis o LevelDB) demuestra la viabilidad y fiabilidad de las estructuras probabilísticas.

9. Referencias (APA 7)

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
- Goodrich, M. T., Tamassia, R., & Mount, D. M. (2011). *Data structures and algorithms in C++* (2nd ed.). Wiley.
- Pugh, W. (1990). Skip lists: A probabilistic alternative to balanced trees. *Communications of the ACM*, 33(6), 668–676. <https://doi.org/10.1145/78973.78977>
- Redis. (2026). *Redis sorted sets*. <https://redis.io/docs/data-types/sorted-sets/>
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
- cppreference.com. (2026). *std::list* — C++ reference. <https://en.cppreference.com/w/cpp/container/list>

10. Anexo — Pseudocódigo completo de una Skip List

```
import random

class NodoSkip:
    def __init__(self, dato, nivel):
        self.dato = dato
        self.siguiente = [None] * (nivel + 1)

class SkipList:
    def __init__(self, p=0.5, max_nivel=32):
        self.cabeza = NodoSkip(None, max_nivel)
        self.p = p
        self.max_nivel = max_nivel
        self.nivel_actual = 0

    def _nivel_aleatorio(self):
        nivel = 0
        while random.random() < self.p and nivel < self.max_nivel:
            nivel += 1
        return nivel

    def buscar(self, dato):
        actual = self.cabeza
        for i in range(self.nivel_actual, -1, -1):
            while actual.siguiente[i] and actual.siguiente[i].dato < dato:
                actual = actual.siguiente[i]
            actual = actual.siguiente[0]
        return actual if actual and actual.dato == dato else None

    def insertar(self, dato):
        actual = self.cabeza
        update = [None] * (self.max_nivel + 1)
```

```
for i in range(self.nivel_actual, -1, -1):
    while actual.siguiete[i] and actual.siguiete[i].dato < dato:
        actual = actual.siguiete[i]
    update[i] = actual

nuevo_nivel = self._nivel_aleatorio()
if nuevo_nivel > self.nivel_actual:
    for i in range(self.nivel_actual + 1, nuevo_nivel + 1):
        update[i] = self.cabeza
    self.nivel_actual = nuevo_nivel

nuevo = NodoSkip(dato, nuevo_nivel)
for i in range(nuevo_nivel + 1):
    nuevo.siguiete[i] = update[i].siguiete[i]
    update[i].siguiete[i] = nuevo

def eliminar(self, dato):
    actual = self.cabeza
    update = [None] * (self.max_nivel + 1)

    for i in range(self.nivel_actual, -1, -1):
        while actual.siguiete[i] and actual.siguiete[i].dato < dato:
            actual = actual.siguiete[i]
        update[i] = actual

    objetivo = actual.siguiete[0]
    if objetivo and objetivo.dato == dato:
        for i in range(self.nivel_actual + 1):
            if update[i].siguiete[i] != objetivo:
                break
            update[i].siguiete[i] = objetivo.siguiete[i]

        while self.nivel_actual > 0 and
self.cabeza.siguiete[self.nivel_actual] is None:
            self.nivel_actual -= 1
        return True
    return False
```